```
        prometheus.io/scrape: "true"                  ❶
        prometheus.io/path: /actuator/prometheus      ❷
        prometheus.io/port: "9001"                    ❸
...
```

❶ Signals that Prometheus should scrape containers in this Pod

❷ Identifies the HTTP endpoint that exposes Prometheus metrics

❸ Specifies the port number where the metrics endpoint is available

Annotations in Kubernetes manifests should be of type `String`, which is why quotes are needed in the case of values that could be mistakenly parsed as numbers or Boolean.

Go ahead and configure metrics and Prometheus for all the remaining applications in the Polar Bookshop system, including the configuration for the Kubernetes manifests. As a reference, you can look at the source code repository accompanying this book (Chapter13/13-end).

The next section will cover another type of telemetry we need in order to monitor applications and make them observable: traces.

13.4 Distributed tracing with OpenTelemetry and Tempo

Event logs, health probes, and metrics provide a wide variety of valuable data for inferring the internal state of an application. However, none of them consider that cloud native applications are distributed systems. A user request is likely to be processed by multiple applications, but so far we have no way to correlate data across application boundaries.

A simple way to solve that problem could be to generate an identifier for each request at the edge of the system (a *correlation ID*), use it in event logs,